

Execution-Path- and Restriction-Aware Autotuning for MPI Collective Operations

Kunting Qi

*Department of Computer Science
University of Illinois Chicago
Chicago, IL, USA
kqi@uic.edu*

Hui Zhou

*Argonne National Laboratory
Lemont, IL, USA
zhouh@anl.gov*

Ke Fan

*Department of Computer and
Information Sciences
Temple University
Philadelphia, PA, USA
ke.fan@temple.edu*

Michael E. Papka

*Argonne National Laboratory
Lemont, IL, USA
Department of Computer Science
University of Illinois Chicago
Chicago, IL, USA
papka@anl.gov*

Sidharth Kumar

*Department of Computer Science
University of Illinois Chicago
Chicago, IL, USA
sidharth@uic.edu*

Rajeev Thakur

*Argonne National Laboratory
Lemont, IL, USA
thakur@anl.gov*

Abstract—Machine-learning-based autotuning is an established approach for selecting MPI collective algorithms at runtime: candidate algorithms are benchmarked across execution configurations, and models are trained to predict the fastest choice for each configuration. However, existing approaches typically assume that every candidate algorithm is valid in all configurations and that the training data accurately reflect each algorithm’s actual performance. These assumptions often fail in production MPI implementations: some algorithms carry configuration-dependent restrictions, invalid requests may silently fall back to other execution paths, and compositional algorithms invoke nested collective operations whose performance depends on the very model being tuned. These issues affect the quality of training data and hence the efficacy of the learned models. We present an execution-path- and restriction-aware autotuning framework, using `MPI_Bcast` in `MPICH` as a case study. The framework trains submodels for the distinct model spaces carved out by configuration restrictions and stitches the submodels together into a holistic runtime selection engine. We further use a multistage training approach. An initial stage measures the algorithms that do not depend on the Broadcast selection model and generates an intermediate model. A subsequent stage benchmarks the compositional algorithms with their internal operations selected by this intermediate model, and the combined data yield a final model that can faithfully select compositional algorithms. On the Polaris system at the Argonne Leadership Computing Facility, the intermediate model alone improves the performance of `smpr`, one of the compositional algorithms, by up to $2.28\times$, and the final model achieves speedups of up to $30.71\times$ over default `MPICH` and $2.76\times$ over previous work, `ACCLiM`.

Index Terms—MPI, collective communication, Broadcast, autotuning, algorithm selection, performance modeling, execution-path awareness

I. INTRODUCTION

MPI collective communication is widely used in scientific computing, data analytics, and distributed machine learning.

At scale, selecting the appropriate collective algorithm is critical to end-to-end application performance and scalability [1], [2]. For a given collective operation, the fastest algorithm can change with message size, communicator size, processes per node, process placement, and hardware platform. Production MPI implementations, such as `MPICH` [3], therefore need to select an appropriate collective algorithm at run time.

Existing collective-tuning work spans several approaches. Vadhiyar et al. automatically tune collective operations by empirically searching over algorithms and segment sizes [4]. Faraj and Yuan combine topology-aware algorithm generation with empirical tuning to achieve high-performance collective communication routines customized for specific platforms [5]. Their follow-up work, `STAR-MPI`, extends this static tuning to dynamic runtime tuning [6]. Pješivac-Grbović et al. learn decision trees from measured performance data to select collective algorithms [7]. Nuriyev and Lastovetsky derive a runtime selection method from code-aware analytical performance models of individual algorithms [8]. Hunold et al. benchmark the available algorithms and train supervised models to predict their runtimes [9], [10]. Hunold et al. later shift to online tuning with `OMPICollTune`, which samples alternative algorithms during application runs and incrementally refines its algorithm-selection probabilities from the observed timings [11]. Qi et al. extend machine-learning-driven autotuning to the non-uniform all-to-all collective `MPI_Alltoallv` [12]. Beyond collectives, Kumar et al. use machine learning to model the performance of the `PIDX` parallel I/O library and select tunable parameters such as aggregator count and aggregation strategy [13].

Wilkins et al. progressively address the training cost that renders ML autotuners impractical at scale. `FACT` combines

domain-informed feature scaling, active learning, and hyperparameter tuning to reduce training data collection time by $6.88\times$ [14]. Its following work, ACCLAiM [15], [16], the current state of the art and our main baseline, further improves the benchmarking efficiency by adopting jackknife-variance point selection, non-power-of-two sampling, a test-set-free convergence criterion, and topology-aware parallel benchmarking, and is the first ML collective autotuner demonstrated to accelerate applications on a production supercomputer despite per-job retraining. However, ACCLAiM tunes only the top-level algorithm selection: it treats each algorithm as a black box, unaware of the internal execution paths of composed algorithms or the constraints that restrict which selections are valid — factors we show dominate performance in several regimes.

As HPC systems become increasingly hybrid, combining multicore nodes, deep memory hierarchies, and accelerators, MPI implementations are adding specialized collective algorithms that exploit these architectural features [17]–[19]. Such algorithms typically carry restrictions on when they can be applied. For example, the `release-gather` algorithm in MPICH is a shared-memory-based algorithm: it can be used only when all processes reside in a shared-memory domain and MPICH is configured with the shared-memory module. Similarly, GPU-specific algorithms can be applied only when their assumptions about memory location hold.

Prior collective autotuning work generally does not account for these restrictions. Such approaches assume a uniform configuration space in which every candidate algorithm is valid at every point, and they assume that the data collected for an algorithm represent the runtime performance of that algorithm. When restrictions are ignored, one of two failures follows: restricted algorithms are excluded from tuning entirely, forfeiting their performance advantages in the regions where they excel, or they are benchmarked outside their valid domains, where a silent fallback attributes another algorithm’s performance to their name. Both cases produce a suboptimal selection policy.

Hybrid architectures also motivate a class of collective algorithms that are compositional in nature [20], [21]. For example, the `sm` algorithm in MPICH decomposes a collective into internode and intranode steps, each of which invokes a collective operation on a subcommunicator. Its performance therefore depends on its nested subcollectives, whose algorithms are chosen by the selection policy being tuned. A training method that ignores this circular dependency measures `sm` under an arbitrary internal selection, and the resulting model is tuned against measurements that do not reflect its deployed behavior. To address these problems, we propose an execution-path- and restriction-aware autotuning framework. We present the framework through `MPI_Bcast` in MPICH, chosen because it is among the most heavily used collectives on production systems [22] and its hierarchical `sm` algorithm exhibits exactly the nested, constrained selection structure our framework targets. Although we use `MPI_Bcast` as the case study, the framework can be adapted to any collective whose algorithms expose internal execution paths and selection constraints, and

we leave other collectives to future work.

Our approach first collects performance data for the algorithms whose measurements carry no circular dependency on the policy being tuned, validating the correctness and execution path of every measurement. Each measurement is annotated with the restrictions of its algorithm, and these restrictions carve the configuration space into separate subspaces. A single restriction, for example, divides the space into two: the configurations that satisfy the restriction and those that do not. For each subspace, we train a submodel on the data valid within it. The final selection model is a composition of these submodels, dispatched at runtime by a first-step restriction check, so that each configuration is served by the submodel trained on its subspace. Within its subspace, each submodel selects the best algorithm among those whose restrictions are satisfied there.

From this initial dataset, we generate an intermediate selection model, which we install as the fixed internal policy of the compositional algorithms. Under this controlled internal selection, we then benchmark the compositional `sm` algorithm, obtaining measurements that reflect its tuned behavior. The validated `sm` measurements are combined with the initial dataset to train the final model, which we compare against default MPICH [3] and ACCLAiM [15]. We implement the proposed framework in MPICH and evaluate it on Polaris, an HPE Apollo 6500 Gen10+ system at the Argonne Leadership Computing Facility, benchmarking nine blocking intracommunicator Broadcast algorithms across 12 process configurations of up to 1024 processes and message sizes from 32 bytes to 32 MiB. Controlling the internal selection of the compositional `sm` algorithm alone improves its performance by up to $2.28\times$ at 768 KiB with 1024 processes and PPN=32, indicating that compositional algorithms must be benchmarked under a tuned internal policy. The final model achieves speedups of up to $30.71\times$ over the default MPICH selection and $2.76\times$ over ACCLAiM [15]. The largest gains come from selecting `sm` where both baselines choose other algorithms. `sm` becomes a candidate at all only once its restrictions and data dependencies are handled explicitly.

II. BACKGROUND AND MOTIVATION

A. MPICH Architecture for Collective Selection

a) From an MPI call to collective selection: When an application invokes a collective operation, MPICH does not immediately execute a communication algorithm. An `MPI_Bcast` call, for example, first passes through argument validation and then enters MPICH’s internal collective layer at `MPICH_Bcast_impl`. There, MPICH constructs a collective signature that identifies the operation and records its parameters, along with derived properties such as message size, communicator size, power-of-two status, and communicator structure. This signature is the input to the Collective SElection (CSEL) engine, which selects the algorithm.

b) Modes of algorithm selection: MPICH supports two selection modes: a direct-algorithm shortcut and automatic CSEL selection. The direct shortcut is used mainly for

```

name=bcast-intra-auto
|-- release-gather condition
|   '-- release-gather
|   comm_size < 8
|   '-- binomial
|-- pof2
|   |-- msg <= 12 KiB -> binomial
|   |-- msg <= 512 KiB -> scatter-RD
|   '-- any -> scatter-ring
'-- any
    |-- msg <= 12 KiB -> binomial
    '-- any -> scatter-ring

```

Fig. 1. Simplified structure of the default MPICH `bcast-intra-auto` CSEL subtree. `scatter-RD` abbreviates `scatter-recursive-doubling-allgather`.

training-data collection and performance diagnosis, whereas automatic selection is the default in production.

In the direct mode, the user sets an MPICH control variable, such as `MPICH_CVAR_BCAST_INTRA_ALGORITHM` for intracommunicator Broadcast, to name a specific algorithm. If the request is valid, MPICH executes the named algorithm directly. Otherwise, it either reports an error or falls through to automatic selection, depending on the configured fallback behavior.

In the automatic mode, MPICH passes the collective signature to its collective-selection infrastructure, CSEL [3], which selects an algorithm according to the call and communicator properties. MPICH also accepts user-provided JSON CSEL rules. A tuning system can therefore benchmark Broadcast algorithms, generate a new selection policy, and deploy it without modifying the application.

c) The CSEL Engine: A CSEL policy is a static JSON decision tree [3]. A simplified example is shown in Fig. 1. The top level routes each collective signature to an operation-specific subtree, and intracommunicator Broadcast uses `name=bcast-intra-auto`. Internal nodes test call properties such as communicator size, message size, power-of-two status, and communicator structure, and traversal ends at an `algorithm=...` leaf naming the selected algorithm and its parameters. An `any` branch provides a fallback when no condition matches.

If CSEL selects a compositional algorithm, each of its internal sub-collectives re-enters CSEL for its own selection. The performance of a compositional algorithm therefore depends on these nested CSEL decisions. In direct mode, forcing a compositional algorithm affects only the top level. Depending on the user’s fallback setting, its sub-collectives either fail or resolve through automatic CSEL selection at each sub-step.

B. MPI_Bcast Algorithms

Table I lists the nine blocking intracommunicator `MPI_Bcast` algorithms considered in this work. The compositional `nb` algorithm depends on the CSEL policy for `MPI_Ibcast`, whereas `smp` depends on the Broadcast policy itself. Besides being limited to intranode execution, `release-gather` requires MPICH to be built with the CH4 POSIX module.

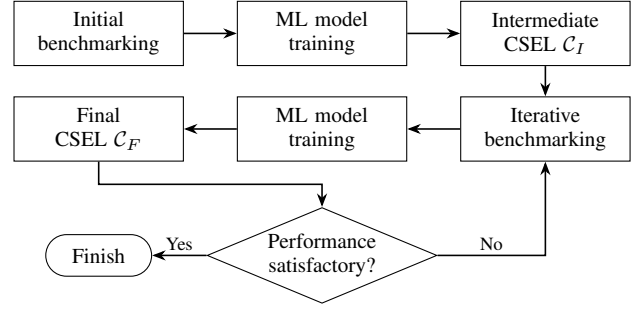


Fig. 2. Restriction- and dependency-aware tuning workflow. Algorithms that do not depend on the policy under construction yield the intermediate policy C_I . Compositional algorithms are then benchmarked under C_I to produce the final policy C_F , and the loop repeats if C_F does not perform satisfactorily.

C. Random Forest and Multilayer Perceptron Models

Random Forest (RF) averages the outputs of decision trees trained on bootstrap samples with random feature subsets. It captures nonlinear effects and feature interactions with little preprocessing, but its piecewise-constant predictions cannot extrapolate beyond the range of the training targets. A Multilayer Perceptron (MLP) is a feed-forward neural network whose smooth functional form follows overall trends and is not restricted to the range of the training targets, at the cost of greater sensitivity to feature scaling, architecture, and regularization [27]. Because the two families complement each other, we train both and keep the more accurate one for each runtime region.

III. METHODOLOGY

A. Method Overview

A naïve tuner assumes that every algorithm is valid everywhere and that each algorithm’s performance can be measured in isolation. However, neither assumption holds here. Some algorithms are valid only in part of the configuration space, such as `release-gather`, and compositional algorithms such as `smp` call other algorithms internally, so their performance depends on the selection policy being tuned. We address validity with *restriction-aware* benchmarking. Each algorithm is benchmarked only where it is valid, and a separate submodel is trained for each partition of the dataset along those validity boundaries. Composing these submodels ensures that no algorithm is ever selected outside its valid region. The dependency problem is handled by *staged* benchmarking. After all algorithms except `smp` are tuned into an intermediate selection policy, `smp` is measured with that policy in place, so its measurements reflect the algorithms it will actually call at runtime. Fig. 2 shows the overall design.

B. Notation

Throughout this section, p denotes communicator size, q denotes processes per node (PPN), and m denotes message size. We write a runtime configuration as $x = (p, q, m)$ and use a for a Broadcast algorithm. Measured and predicted execution

TABLE I
MPICH BROADCAST ALGORITHMS CONSIDERED IN THIS STUDY. HORIZONTAL RULES SEPARATE THE NON-COMPOSITIONAL, COMPOSITIONAL, AND RESTRICTED ALGORITHMS.

Algorithm	Method	Description
binomial [1]	Direct tree	Forwards the complete message through a binomial tree.
circ-graph [23]	Direct graph	Exchanges message blocks according to a circulant-graph schedule.
scatter-recursive-doubling-allgather [24], [25]	Scatter-allgather	Combines a binomial scatter with a recursive-doubling allgather.
scatter-ring-allgather	Scatter-allgather	Combines a binomial scatter with a ring allgather.
tree	Parameterized tree	Supports configurable tree type, branching factor, and blocking mode.
pipelined-tree	Parameterized pipeline	Pipelines configurable message chunks through a configurable tree.
nb* [26]	Nonblocking composition	Invokes <code>MPI_Ibcast</code> and waits for completion, with the internal algorithm chosen by the <code>Ibcast</code> policy.
smp* [20]	Hierarchical composition	Composes node-root and node-local Broadcast operations whose algorithms are selected through Broadcast CSEL.
release-gather† [19]	Restricted shared memory	Uses CH4 POSIX release/gather synchronization and is valid only for intranode execution.

*Compositional algorithm whose internal collective operations are selected by a CSEL policy.

†Algorithm subject to a configuration-dependent validity restriction.

times are written as $T(x, a)$ and $\hat{T}(x, a)$, respectively. $\mathcal{C}_I$ denotes the intermediate CSEL policy that does not select `smp`, and $\mathcal{C}_F$ denotes the final CSEL policy used for deployment.

C. Restriction-Aware Tuning

An algorithm is benchmarked, modeled, and selected only in configurations where it is valid. We enforce this restriction at every stage of the pipeline.

a) Enforcing restrictions during benchmarking: A benchmark must *fail* on any data point that violates its algorithm’s restrictions, rather than silently returning a measurement. For a directly forced algorithm, we disable top-level fallback so that a restriction violation returns an error instead of diverting to another algorithm, while for a compositional algorithm, we keep fallback enabled so that its nested sub-collectives are resolved by the installed CSEL policy. In both cases, we keep a measurement only if the collective counters confirm that the requested top-level and nested algorithms actually ran. Every retained measurement therefore reflects the algorithm it is labeled with.

b) Restrictions and candidate sets: Each restriction acts as a predicate over the configuration space. The candidate set of a configuration contains every unrestricted algorithm, plus every restricted algorithm whose restrictions the configuration satisfies. Satisfying a restriction therefore *enlarges* the candidate set rather than shrinking it: a single-node shared-memory configuration admits `release-gather` in addition to all unrestricted algorithms, and a hierarchical communicator additionally admits `smp`. Configurations with the same candidate set form a subspace. For each subspace, we assemble a dataset containing only the validated measurements of its candidates.

c) Fitting submodels: We fit a separate submodel to each subspace dataset. Within each subspace, the data are further partitioned by message size at an empirically chosen boundary of 16 KiB. Fig. 3 illustrates the rationale for this

partition. Below 16 KiB, most algorithm curves are relatively flat or exhibit local fluctuations, as latency is dominated by startup and software overhead. Above this boundary, the curves increase smoothly with message size, indicating bandwidth- and data-movement-dominated behavior. Because a single model has difficulty capturing both regimes, we fit both an RF and an MLP for each region and retain the model with the lower validation error. Both models take as input the base-2 logarithms of p , q , and m , together with a one-hot encoding of the algorithm, and predict the logarithm of the latency. The data are split 70/10/20 into training, validation, and test sets, with all algorithms measured at the same runtime configuration x assigned to the same set to prevent information leakage across sets. The RF consists of 200 trees whose maximum depth is selected on the validation set, and the MLP has two ReLU hidden layers with 128 and 64 units, respectively. Prediction error is measured using the mean absolute percentage error (MAPE),

$$\text{MAPE}(f_g) = \frac{100}{|\mathcal{D}_g|} \sum_{(x,a,T) \in \mathcal{D}_g} \left| \frac{\hat{T}_g(x,a) - T(x,a)}{T(x,a)} \right|, \quad (1)$$

which captures the relative difference between the predicted latency $\hat{T}_g(x,a)$ and the measured latency $T(x,a)$ over a dataset $\mathcal{D}_g$ of modeling region g , namely its validation set for model selection and its test set in Table II. The per-region selection is justified empirically: as Table II shows, no single family wins everywhere. MLP is more accurate for both large-message regions, whereas RF is more accurate for both small-message regions, where the fluctuating latency profiles favor its piecewise structure.

d) Composing the final selector: Each submodel is compiled into its own CSEL subtree covering only its subspace and valid algorithms. We assemble the final selector by placing these subtrees beneath restriction-checking nodes: at runtime,

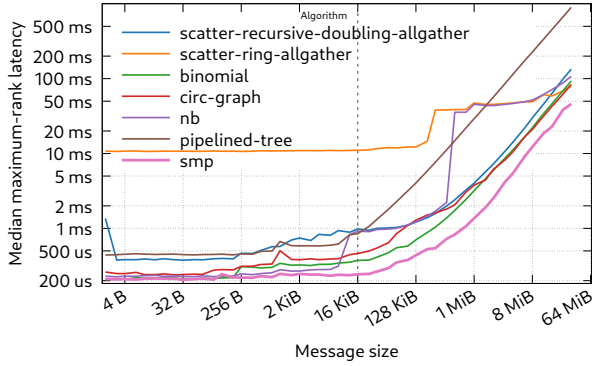


Fig. 3. Representative message-size performance for 1024 processes at PPN = 32. The dashed line marks the empirical 16 KiB boundary used for model fitting.

TABLE II

TEST-SET MAPE OF THE REGION-SPECIFIC RUNTIME MODELS BY NODE COUNT AND MESSAGE SIZE. BOLD MARKS THE BETTER OF RF AND MLP. THE MODEL SELECTED ON THE VALIDATION SET (LAST COLUMN) MATCHES THE BETTER PERFORMER IN ALL FOUR REGIONS.

Region	RF (%)	MLP (%)	Selected
Single / small	4.65	7.85	RF
Single / large	28.46	7.32	MLP
Multinode / small	4.34	6.69	RF
Multinode / large	16.75	4.90	MLP

the tree first tests the communicator context to identify the subspace, then descends into the corresponding subtree. Consequently, a configuration is served only by the submodel trained on its subspace, and no algorithm can be selected outside its valid domain.

D. Staged Tuning for Compositional Algorithms

The restriction-aware pipeline above assumes each measurement reflects a fixed algorithm. This does not hold for `smp`, whose internal Broadcasts re-enter Broadcast CSEL: its measured performance depends on the very policy being tuned. We break this circular dependency by tuning in stages.

a) Stage 1: non-compositional tuning: The initial dataset excludes `smp` and contains only the non-compositional algorithms and `nb`. Although `nb` is compositional, it invokes `Ibcast` rather than `Broadcast`, and the `Ibcast` policy is held fixed throughout. Its measurements therefore carry no circular dependency and belong in this stage. Applying the restriction-aware pipeline of Section III-C to this dataset yields the intermediate policy $\mathcal{C}_I$. Because `smp` is absent from the candidate set, $\mathcal{C}_I$ cannot select `smp` for an internal Broadcast, which would otherwise create an unbounded recursive selection path.

b) Stage 2: compositional tuning: We install $\mathcal{C}_I$ as the Broadcast policy and benchmark `smp`, now allowing its internal sub-collectives to resolve through CSEL (that is, through $\mathcal{C}_I$). Each `smp` measurement therefore reflects a fixed, known internal selection rather than an untuned default, and the circular dependency is resolved for the duration of the stage. We validate these measurements with the same counter-based

path checks and add the valid ones to the Stage-1 data, forming the integrated dataset. Re-applying the restriction-aware pipeline to this larger dataset, now including `smp` in the hierarchical subspace, yields the final policy $\mathcal{C}_F$.

c) Convergence: $\mathcal{C}_F$ may select different internal Broadcasts than $\mathcal{C}_I$ did during Stage 2, so the `smp` measurements are, in principle, tied to the intermediate policy under which they were collected. We therefore evaluate $\mathcal{C}_F$ in a separate end-to-end run (Section V) to confirm that its deployed performance is satisfactory. If it were not, the procedure could iterate, installing $\mathcal{C}_F$ as the new internal policy, re-measuring `smp`, and regenerating the policy, until the selected algorithms stop changing between rounds. One iteration proved sufficient in our experiments, likely because only one compositional algorithm has a circular dependency. More iterations may be necessary when several such algorithms are present.

IV. EXPERIMENTAL SETUP

A. Experimental Platform

We conducted experiments on Polaris, an HPE Apollo 6500 Gen10+ system at the Argonne Leadership Computing Facility. Each compute node is equipped with one 2.8GHz AMD EPYC 7543P processor containing 32 physical cores and 512 GiB of DDR4 memory. The compute nodes are connected through the HPE Slingshot 11 interconnect. Each node has two 200 Gbps Cassini network interfaces, providing an aggregate injection bandwidth of 400 Gbps.

B. Benchmarking Configuration Space

The data used to train the performance models cover the following configurations, in terms of the number of processes per node (PPN) and the total process count p :

- Process configurations: We cover five PPN values (1, 4, 8, 16, and 32) and six node counts (1, 2, 4, 8, 16, and 32 while PPN=1 starts at two nodes), yielding configurations with up to 1,024 processes.
- Message sizes: 49 sizes from 2 B to 32 MiB, comprising every power of two and one intermediate size between each adjacent pair (2, 3, 4, 6, 8, 12, ... bytes).
- Root and placement: rank 0 is the Broadcast root. Nodes are allocated with the `PBS place=scatter` policy, and processes are evenly distributed across them and bound to distinct physical cores.

The PPN=1 configurations cover the node-root Broadcasts invoked by compositional algorithms. The remaining configurations cover both single-node and multinode executions used to fit the regional performance models.

C. Evaluation Configuration

The end-to-end evaluation uses a smaller regular grid that is distinct from the benchmarking space. It contains 21 power-of-two message sizes from 32 B to 32 MiB and 12 process configurations formed by 4, 8, 16, and 32 nodes at PPN values of 8, 16, and 32 (from $p = 32$ to $p = 1024$), for a total of 252 configurations (p, q, m). The Broadcast root and node placement are the same as in the benchmarking runs.

D. Baselines

We compare the final CSEL policy $\mathcal{C}_F$ against two baselines.

a) *MPICH default auto*: The first baseline is the unmodified automatic Broadcast policy distributed with the evaluated MPICH build. We set `MPICH_CVAR_BCAST_INTRA_ALGORITHM` to `auto` and do not load a user-provided CSEL JSON file. MPICH therefore traverses its built-in default CSEL rules to select the Broadcast algorithm for each runtime configuration [3].

b) *ACCLaIM-generated CSEL*: The second baseline is a CSEL policy produced by ACCLaIM [15]. We run the ACCLaIM Broadcast tuning workflow on Polaris and load its generated static CSEL JSON file through MPICH’s CSEL interface. In the end-to-end results, *ACCLaIM* denotes collective execution under this generated policy.

E. Measurement Methodology

Each timed iteration starts with an `MPI_Barrier`, times one `MPI_Bcast` on every rank, and records the maximum time across ranks. For the training data, each combination of algorithm, process configuration, and message size runs as many warm-up iterations as measured iterations. The number of measured iterations ranges from 20 to 250, with fewer iterations at larger scales, and the median over the measured iterations serves as the training target data. In the end-to-end comparison, each point runs 100 iterations, and we report the median of the second half, treating the first half as warm-up.

All three policies use the same MPICH build, benchmark executable, process placement, and message-size sequence, and run sequentially within the same PBS allocation on the same nodes, so only the active CSEL rules differ. No policy selects algorithms based on the root, so using rank 0 as the root affects the three policies equally. Because MPI timings can vary between separate executions even under identical settings [28], [29], we repeat several small-message comparisons with 100,000 iterations in five separate jobs (Table V).

V. RESULTS

A. Algorithm Performance Data

We first examine representative benchmark measurements to identify recurring performance patterns among commonly used `MPI_Bcast` algorithms. These results also demonstrate why practical autotuning must account for not only the relative performance of candidate algorithms, but also their applicability restrictions and execution-path dependencies.

Fig. 4 compares three algorithms at $p = \text{PPN} = 32$, where `release-gather` is applicable. `release-gather` outperforms the other algorithms for all tested message sizes below 32 MiB, followed by `binomial`, while `scatter-ring-allgather` performs worst, particularly for small messages. This result shows that restricted algorithms can provide valuable performance opportunities when their restriction is satisfied and therefore should not be excluded merely to simplify benchmarking.

Fig. 5 shows that `binomial` consistently outperforms `scatter-ring-allgather` until the message becomes

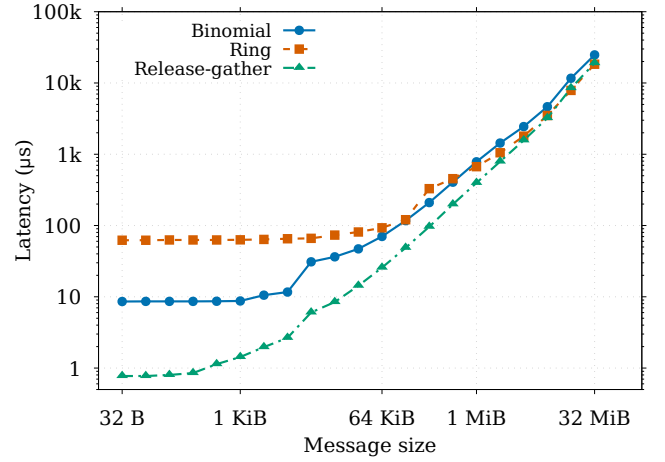


Fig. 4. Selected single-node `MPI_Bcast` algorithm performance at $p = 32$ and $\text{PPN} = 32$. Ring denotes `scatter-ring-allgather`.

sufficiently large for the latter to become competitive. However, this crossover point shifts toward larger message sizes as the process count increases. Thus, the preferred algorithm depends jointly on message size and process count rather than on a fixed message-size threshold.

Fig. 6 compares three direct algorithms as the process count increases from 256 to 1024. `binomial` generally achieves the lowest latency for small and medium messages, while `scatter-recursive-doubling-allgather` incurs higher latency and `scatter-ring-allgather` becomes competitive only for sufficiently large messages. The gaps and crossover points again shift with process count.

B. Effect of the Intermediate CSEL on `smp`

A central claim of this work is that compositional algorithms must be benchmarked under a tuned internal selection policy. Because the internal Broadcasts of `smp` are resolved through the installed CSEL policy, the same top-level algorithm can perform differently depending on which internal algorithms are selected. If this effect is significant, measurements of `smp` collected under the untuned default policy would misrepresent its achievable performance and mislead the final model, making the staged approach necessary. This subsection quantifies the effect directly.

To isolate this effect, we fix the top-level algorithm to `smp` and vary only the policy that selects its internal Broadcast algorithms, comparing the MPICH default CSEL with $\mathcal{C}_I$. Both configurations are evaluated at $\text{PPN}=32$ with process counts from 32 to 1024, using the same 49 message sizes from 2 bytes to 32 MiB at each process count. Because the top-level algorithm, process configuration, and message sizes are identical, any performance difference between the two runs can be attributed to the internal algorithm selection. We define the speedup as

$$S_{\text{smp}}(x) = \frac{T_{\text{smp}}(x; \mathcal{C}_{\text{default}})}{T_{\text{smp}}(x; \mathcal{C}_I)}, \quad (2)$$

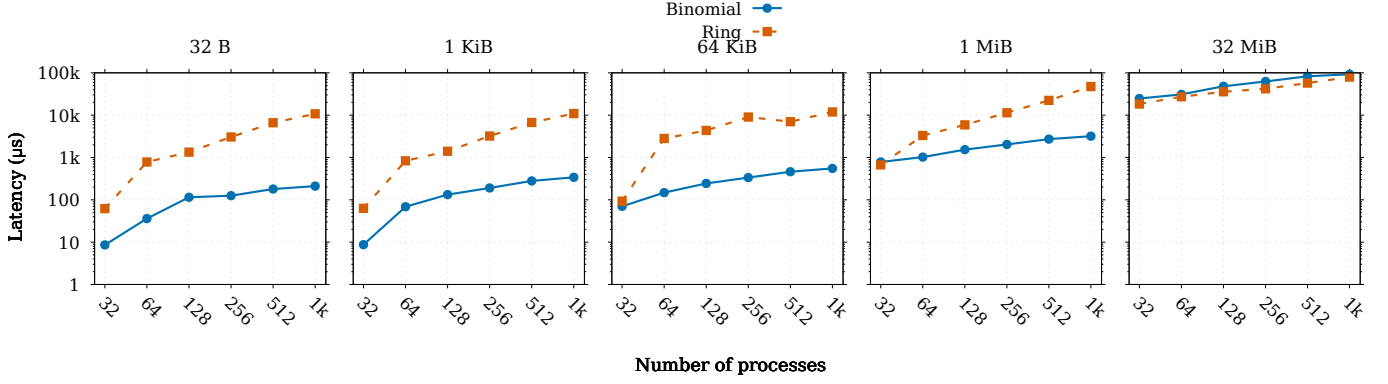


Fig. 5. Binomial and ring Broadcast latency as communicator size increases at PPN = 32 for five message sizes.

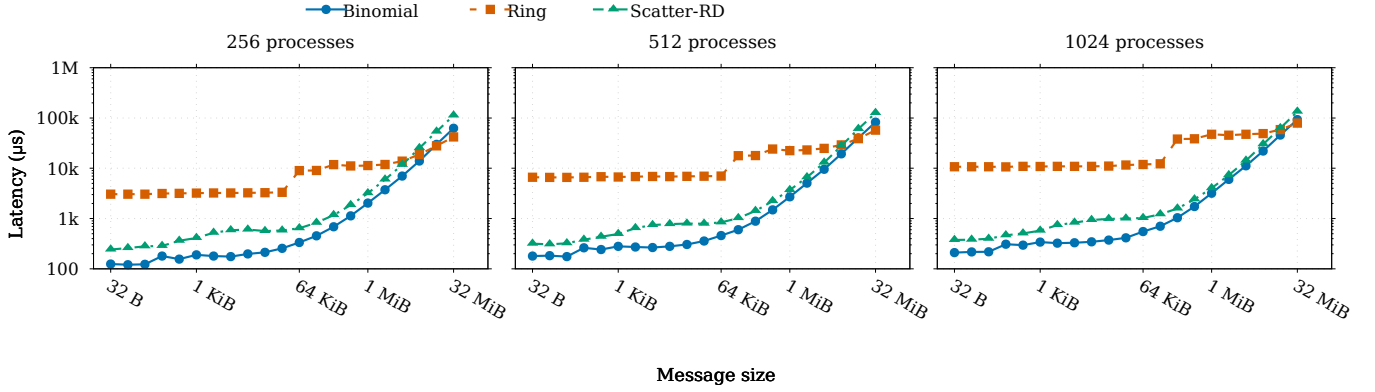


Fig. 6. Binomial, ring, and scatter-recursive-doubling-allgather (scatter-RD) Broadcast latency at PPN = 32 for $p \in \{256, 512, 1024\}$. All panels share the same logarithmic latency axis.

where $S_{\text{smp}}(x) > 1$ indicates that the intermediate policy is faster at configuration x .

As shown in Fig. 7, smp under C_I is faster for 43 of the 49 message sizes at $p = 1024$ and PPN=32. Across tested configurations, the largest speedup is $2.28\times$, with message size $m = 768$ KiB, $p = 1024$ and PPN=32.

These results support the need for initial and iterative benchmarking. The performance of smp depends on its internal CSEL decisions. It should therefore be measured during iterative benchmarking, after its internal selection environment has been fixed.

C. End-to-End Performance of the Final CSEL

We compare the final CSEL policy C_F with MPICH default auto and ACCLAI M using separate end-to-end MPI executions. A runtime configuration is retained only when all three policies pass the communication-correctness checks and produce complete timing and algorithm-trace records.

The comparison covers 12 process configurations and 21 message sizes, resulting in 252 paired evaluation points. Unlike the offline model evaluation, these experiments measure the complete deployed execution path, including CSEL

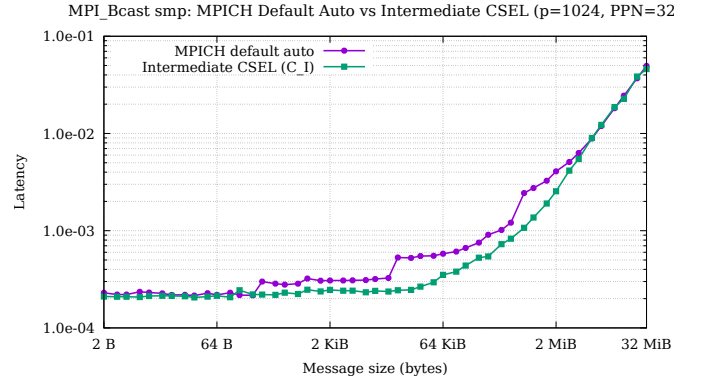


Fig. 7. smp latency under the default internal policy and C_I for $p = 1024$ and PPN=32.

traversal, top-level algorithm selection, the internal recursive Broadcast selections made by compositional algorithms such as smp , and the complete collective execution.

For a baseline policy $\mathcal{B}$, we define the end-to-end speedup

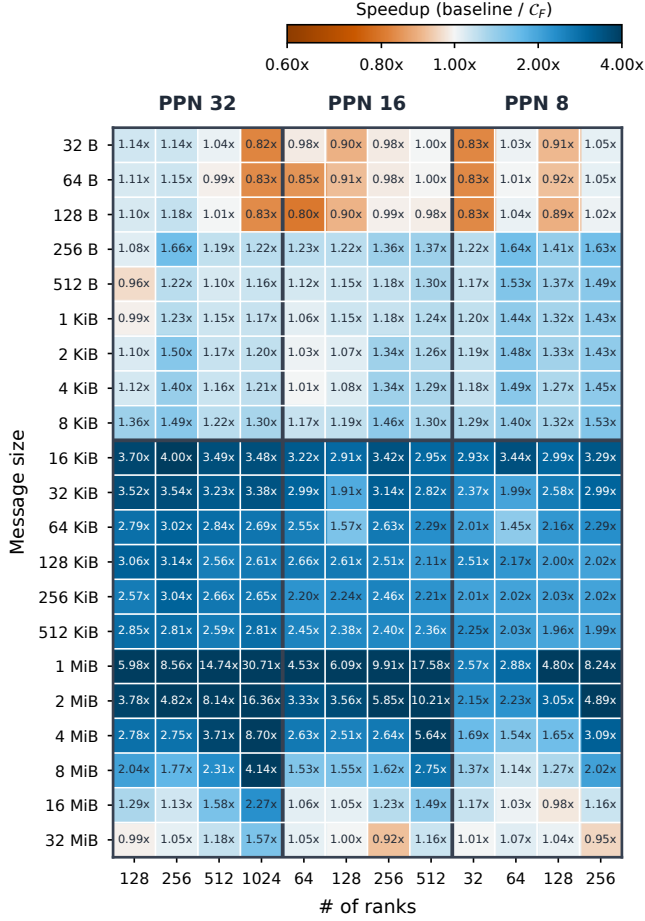


Fig. 8. Speedup of the final CSEL policy over MPICH default auto, computed as baseline latency divided by final-policy latency. Values above $1\times$ favor the final policy. Cells from 32 B to 4 KiB at 4 nodes come from the 100,000-iteration reruns (Set 5 in Table V), and all other cells come from 100-iteration runs.

of the final policy as

$$S_B(x) = \frac{T_B(x)}{T_{C_F}(x)}, \quad (3)$$

where $x = (p, q, m)$ denotes the runtime configuration. A value greater than one indicates that C_F is faster than the baseline, whereas a value below one indicates a regression.

Compared to MPICH default auto, C_F is faster at 225 of the 252 configurations (89.3%) and achieves a geometric-mean speedup of $1.81\times$. Results are mixed from 32–128 B, with the minimum speedup of $0.80\times$ at PPN=16, 64 ranks, and 128 B. All 12 configurations improve at 256 B and from 2 KiB through 8 MiB, and only one configuration regresses at each of 512 B and 1 KiB. The largest improvement is $30.71\times$ at PPN=32, 1024 ranks, and 1 MiB. Gains diminish at 16–32 MiB, where several configurations approach parity (Fig. 8).

Relative to ACCLAiM, C_F is faster at 207 of the 252 points (82.1%) and achieves a geometric-mean speedup of $1.33\times$.

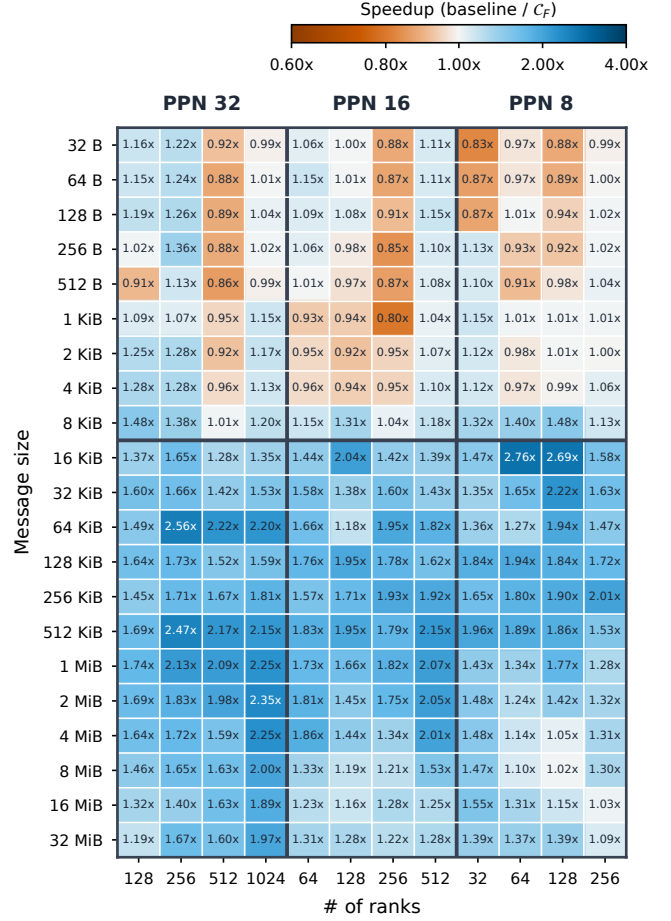


Fig. 9. Speedup of the final CSEL policy over ACCLAiM, computed and sourced as in Fig. 8.

TABLE III
BENCHMARKED, PREDICTED, AND DEPLOYED EVALUATION LATENCIES (ms) FOR THE SUCCESSFUL 1 MiB CASE AT PPN=32 AND $p = 1024$.

Algorithm	Benchmark	Prediction	Evaluation
smp	1.37	1.36	1.48
binomial	3.19	3.30	3.33
scatter-ring-allgather	47.43	43.90	45.47

Results are configuration-dependent below 8 KiB, reaching a minimum of $0.80\times$ at PPN=16, 256 ranks, and 1 KiB. Every tested configuration from 8 KiB through 32 MiB favors C_F . The maximum speedup is $2.76\times$ at PPN=8, 64 ranks, and 16 KiB (Fig. 9).

To connect these aggregate patterns to concrete deployed selections, we analyze two configurations that represent the contrasting outcomes in the heatmaps: a large-message case in which C_F outperforms both baselines and a small-message case in which it underperforms both.

a) *Representative outperformance case:* At PPN=32, 1024 ranks, and a 1 MiB message, the three policies select

TABLE IV

BENCHMARKED, PREDICTED, AND DEPLOYED EVALUATION LATENCIES (μs) FOR THE 32 B REGRESSION AT PPN=8 AND $p = 32$. THE TWO BINOMIAL EVALUATION VALUES CORRESPOND TO MPICH DEFAULT AUTO AND ACCLAiM, RESPECTIVELY.

Algorithm	Benchmark	Prediction	Evaluation
nb	58.83	59.25	68.42
binomial	59.51	60.29	56.65/56.60

different top-level algorithms. The final policy selects `smp`, MPICH default auto selects `scatter-ring-allgather`, and ACCLAiM selects `binomial`. MPICH default auto follows hand-written heuristics that assume `scatter-ring-allgather` suits large messages. ACCLAiM does not perform staged training, so `smp` is misrepresented in its dataset and is not selected. The resulting speedups are $30.71\times$ over MPICH default auto and $2.25\times$ over ACCLAiM. Table III reports the benchmarked, predicted, and deployed evaluation latencies from our model. The model preserved the benchmarked ordering, and deployment remained consistent with it. This stable ordering confirms that the selection of `smp` was not an algorithm-trace artifact.

This case also demonstrates the benefit of staged benchmarking. Fixing the node-local and node-root Broadcast selections with intermediate CSEL policy $\mathcal{C}_I$ made the complete `smp` path a validated, competitive algorithm candidate and revealed a performance opportunity missed by both baselines.

b) Representative underperformance case: At PPN = 8, $p = 32$, and $m = 32$ B, the final policy selects `nb`, whereas both MPICH default auto and ACCLAiM select `binomial`. The second-half median latencies are $68.42 \mu s$ for the final policy, $56.65 \mu s$ for MPICH default auto, and $56.60 \mu s$ for ACCLAiM. These results correspond to speedups of $0.83\times$ and $0.83\times$ over MPICH default auto and ACCLAiM, respectively, making this a representative small-message regression in the evaluation results.

To diagnose this regression, Table IV compares the benchmarked, predicted, and deployed evaluation latencies of the selected algorithms. In the benchmark data, `nb` outperforms `binomial` by only 1.1%. Similarly, the multinode-small RF model predicts only a 1.7% advantage for `nb`. During deployment, however, the ordering reverses: the final policy using `nb` is about 21% slower than both baseline policies using `binomial`.

Because MPICH default auto and ACCLAiM select the same top-level `binomial` algorithm and achieve nearly identical latencies, the regression is associated with the final policy’s selection of `nb`. Its small advantage in the benchmark data and model prediction does not persist during deployment.

To determine whether this reversal represents a broader risk, Table V reports five repeated measurements for several small-message configurations.

At PPN = 32, $p = 128$, and $m = 32$ B, the measured speedup over MPICH ranges from $0.94\times$ to $1.37\times$, while that over ACCLAiM ranges from $0.74\times$ to $1.54\times$. Similarly, at

TABLE V

SPEEDUP OF THE FINAL CSEL POLICY OVER MPICH DEFAULT AUTO AND ACCLAiM IN FIVE REPEATED 100,000-ITERATION RUNS.

Baseline	Set 1	Set 2	Set 3	Set 4	Set 5
PPN=32, $p = 128$, $m = 32$ B					
MPICH	1.02	1.22	0.94	1.37	1.14
ACCLAiM	0.83	1.15	0.74	1.54	1.16
PPN=16, $p = 64$, $m = 128$ B					
MPICH	1.51	1.38	1.40	0.77	0.80
ACCLAiM	1.31	1.50	0.98	0.75	1.09
PPN=8, $p = 32$, $m = 256$ B					
MPICH	1.47	1.28	1.41	1.37	1.22
ACCLAiM	1.38	1.34	1.48	1.32	1.13
PPN=32, $p = 128$, $m = 4$ KiB					
MPICH	1.12	1.27	1.29	1.26	1.12
ACCLAiM	1.06	1.38	1.15	1.47	1.28

PPN = 16, $p = 64$, and $m = 128$ B, the measured ordering alternates between speedup and slowdown. In contrast, the final policy remains faster across all five result sets for the 256 B and 4 KiB configurations, where its speedups over both MPICH and ACCLAiM remain above $1\times$. These results show that small-message performance is not universally unstable, and run-to-run variation can reverse the measured ordering when competing policies have similar latency. The 32 B `nb` regression therefore illustrates the risk of selecting an algorithm based on a small predicted advantage in the small-message scenario, where performance differences observed during benchmarking and modeling may not persist at runtime.

This case indicates that direct minimum-latency selection may be insufficiently robust when the predicted performance separation is small. A more robust policy could require a candidate to outperform the MPICH default algorithm by a prediction margin before replacing it. For example, a candidate a_{new} could replace the default algorithm a_{default} only when

$$\hat{T}(x, a_{\text{new}}) < (1 - \delta)\hat{T}(x, a_{\text{default}}), \quad (4)$$

where $\delta > 0$ is a validation-selected margin.

Such a guard would retain the default algorithm for small predicted gaps while preserving selections with a large predicted advantage, such as the successful 1 MiB case. The margin must be selected using validation data and evaluated in new MPI jobs to avoid tuning and reporting on the same measurements. We therefore defer a sensitivity study of δ to future work.

Together, these cases show both sides of learned collective selection. When the training-time performance separation is large and remains stable, the staged inclusion of `smp` exposes substantial performance gains missed by the baseline policies. When the predicted separation is small, variation between the training and deployment profiles can reverse the algorithm ordering and cause a local regression.

The end-to-end results therefore support the central motivation for dependency-aware staged benchmarking: compositional algorithms such as `smp` can provide large gains, but they must first be measured under a controlled internal selection policy. At the same time, the small-message case shows that

the final policy, which selects the algorithm with the minimum predicted latency, should be augmented with a margin-based guard as in (4) when predicted differences are small.

VI. DISCUSSION, LIMITATIONS, AND FUTURE WORK

a) Dependency-aware and region-specific tuning: Our results show that MPI collective algorithms are not always independent. Some algorithms call other collective paths internally. Their performance therefore depends on the internal selection policy. The proposed workflow of initial and iterative benchmarking first fixes these internal dependencies and then measures the complete top-level execution. In this way, algorithm restrictions and recursive dependencies are included explicitly in benchmarking, model fitting, and CSEL generation.

The experiments also show that one global model may not represent all performance patterns in the runtime space. We therefore select models separately for single-node and multi-node configurations, and for small and large messages. These regional models form the final model and its CSEL policy C_F . The current prototype uses a fixed empirical 16 KiB boundary. Future work can determine the number of regions and their boundaries automatically through change-point detection or validation-based partitioning.

b) Robust and consistent deployment: Prediction errors in a learned policy may cause local suboptimal selections. Future work can introduce a prediction margin. A new algorithm would replace the MPICH default choice only when its predicted advantage is large enough. Otherwise, the default policy would be retained. The margin and other safety rules should be selected using validation data. They should then be evaluated using new MPI jobs to avoid tuning on the test set.

In addition, the `smp` measurements from iterative benchmarking are collected under the intermediate CSEL policy C_I , while final deployment uses the newly generated C_F , and the two policies may make different internal Broadcast selections. Consequently, our final performance conclusions rely on paired end-to-end experiments after C_F is installed rather than on offline model error alone. This work does not explicitly solve for a stable policy between the two CSEL rules. Future work can repeat benchmarking, model refitting, and CSEL generation until the selected algorithms stop changing between consecutive rounds. Another option is to keep the recursive subtrees generated by initial benchmarking in the final CSEL policy and update only the top-level selection rules.

c) Reducing benchmarking cost and automation: The current method measures every candidate algorithm across process counts, PPN values, and message sizes. On Polaris, benchmarking all configurations up to 32 nodes took approximately 108 node-hours (97 for initial and 11 for iterative benchmarking), whereas ACCLaIM's active-learning workflow converged after 500.8 s on a 32-node allocation (4.45 node-hours). The cost grows with the size of the search space and with the number of compositional algorithms, which may require additional benchmarking stages. Future work can combine the staged methodology with active learning, prioritizing

runtime conditions with high model uncertainty or similar predicted performance among candidate algorithms. Adaptive message-size sampling and early stopping may further reduce the required node hours.

The current algorithm restrictions and recursive dependencies are identified mainly through MPICH documentation, source-code inspection, and execution counters. Future work can use static analysis to extract CVAR conditions, communicator requirements, and internal collective calls automatically. This information can be used to build a dependency graph and generate the initial and iterative benchmarking procedures, sets of candidate algorithms, and fallback branches.

d) Generality and portability: The current experiments evaluate MPI Broadcast on one platform and keep the parameters of `tree` and `pipelined-tree` fixed. Future work can add tree type, branching factor, chunk size, and other parameters to the algorithm search space. The same workflow can also be extended to Reduce, Gather, Allreduce, Allgather, and other collective operations.

Different platforms may have different algorithm crossover points. The models must therefore be retrained or calibrated for each target platform. Selection models pre-trained across multiple systems, such as PML-MPI [30], transfer learning, a small number of measurements on the target system, and controlled lightweight runtime feedback may reduce cross-platform tuning cost while preserving the low runtime overhead of static CSEL rules.

ACKNOWLEDGMENT

The authors gratefully acknowledge the funding and computing resources that supported this work. This work was funded in part by NSF PPOSS grant CCF-2316157 and NSF SHF grant CCF-2401274. Argonne National Laboratory's work was supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357. We are thankful to the ALCF's Director's Discretionary (DD) program for providing us with compute hours to run our experiments on the Polaris supercomputer located at the Argonne National Laboratory. This research used resources of the Argonne Leadership Computing Facility, which is a U.S. Department of Energy Office of Science User Facility operated under contract DE-AC02-06CH11357.

REFERENCES

- [1] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, Feb. 2005, doi: 10.1177/1094342005051521.
- [2] T. Hoefler, W. Gropp, R. Thakur, and J. L. Träff, "Toward performance models of MPI implementations for understanding application scaling issues," in *Proceedings of the 17th European MPI Users' Group Meeting Conference on Recent Advances in the Message Passing Interface*, ser. EuroMPI'10. Berlin, Heidelberg: Springer-Verlag, 2010, pp. 21–30, doi: 10.1007/978-3-642-15646-5_3.
- [3] MPICH Developers, "Collective selection framework," MPICH Design Documentation, Accessed: Aug. 4, 2026. [Online]. Available: https://github.com/pmodels/mpich/blob/main/doc/wiki/design/Collective_Selection_Framework.md

- [4] S. S. Vadhiyar, G. E. Fagg, and J. Dongarra, "Automatically tuned collective communications," in *Proceedings of the 2000 ACM/IEEE Conference on Supercomputing*, ser. SC '00. USA: IEEE Computer Society, 2000, p. 3, doi: 10.1109/SC.2000.10024.
- [5] A. Faraj and X. Yuan, "Automatic generation and tuning of MPI collective communication routines," in *Proceedings of the 19th Annual International Conference on Supercomputing*, ser. ICS '05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 393–402, doi: 10.1145/1088149.1088202.
- [6] A. Faraj, X. Yuan, and D. Lowenthal, "STAR-MPI: Self tuned adaptive routines for MPI collective operations," in *Proceedings of the 20th Annual International Conference on Supercomputing*, ser. ICS '06. New York, NY, USA: Association for Computing Machinery, 2006, pp. 199–208, doi: 10.1145/1183401.1183431.
- [7] J. Pješivac-Grbović, G. Bosilca, G. E. Fagg, T. Angskun, and J. J. Dongarra, "Decision trees and MPI collective algorithm selection problem," in *Euro-Par 2007 Parallel Processing*, A.-M. Kermarrec, L. Bougé, and T. Priol, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 107–117, doi: 10.1007/978-3-540-74466-5_13.
- [8] E. Nuriyev and A. Lastovetsky, "Efficient and accurate selection of optimal collective communication algorithms using analytical performance modeling," *IEEE Access*, vol. 9, pp. 109 355–109 373, 2021, doi: 10.1109/ACCESS.2021.3101689.
- [9] S. Hunold and A. Carpen-Amarie, "Algorithm selection of MPI collectives using machine learning techniques," in *2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems*. IEEE, 2018, pp. 45–50, doi: 10.1109/PMBS.2018.8641622.
- [10] S. Hunold, A. Bhatlele, G. Bosilca, and P. Knees, "Predicting MPI collective communication performance using machine learning," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 2020, pp. 259–269, doi: 10.1109/CLUSTER49012.2020.00036.
- [11] S. Hunold and S. Steiner, "OMPICollTune: Autotuning MPI collectives by incremental online learning," in *2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. IEEE, Nov. 2022, pp. 123–128, doi: 10.1109/PMBS56514.2022.00016.
- [12] K. Qi, K. Fan, J. Domke, S. Ba, V. Vishwanath, M. E. Papka, and S. Kumar, "Machine learning driven auto-tuning for non-uniform all-to-all collectives," in *2025 IEEE 32nd International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, Dec. 2025, pp. 151–160, doi: 10.1109/HiPC66333.2025.00024.
- [13] S. Kumar, A. Saha, V. Vishwanath, P. Carns, J. A. Schmidt, G. Scorzelli, H. Kolla, R. Grout, R. Latham, R. Ross, M. E. Papka, J. Chen, and V. Pascucci, "Characterization and modeling of PDX parallel I/O for performance optimization," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ser. SC '13. New York, NY, USA: Association for Computing Machinery, 2013, doi: 10.1145/2503210.2503252.
- [14] M. Wilkins, Y. Guo, R. Thakur, N. Hardavellas, P. Dinda, and M. Si, "A FACT-based approach: Making machine learning collective autotuning feasible on exascale systems," in *2021 Workshop on Exascale MPI (ExaMPI)*. IEEE, 2021, pp. 36–45, doi: 10.1109/ExaMPI54564.2021.00010.
- [15] M. Wilkins, Y. Guo, R. Thakur, P. Dinda, and N. Hardavellas, "AC-CLAiM: Advancing the practicality of MPI collective communication autotuning using machine learning," in *2022 IEEE International Conference on Cluster Computing*. IEEE, 2022, pp. 161–171, doi: 10.1109/CLUSTER51413.2022.00030.
- [16] —, "Practical machine learning autotuning for large-scale collective communication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 37, no. 5, pp. 1032–1047, May 2026, doi: 10.1109/TPDS.2026.3661876.
- [17] R. L. Graham and G. Shipman, "MPI support for multi-core architectures: Optimized shared memory collectives," in *Proceedings of the 15th European PVM/MPI Users' Group Meeting on Recent Advances in Parallel Virtual Machine and Message Passing Interface*. Berlin, Heidelberg: Springer-Verlag, 2008, pp. 130–140, doi: 10.1007/978-3-540-87475-1_21.
- [18] A. R. Mamidala, R. Kumar, D. De, and D. K. Panda, "MPI collectives on modern multicore clusters: Performance optimizations and communication characteristics," in *Proceedings of the 2008 Eighth IEEE International Symposium on Cluster Computing and the Grid*, ser. CCGRID '08. USA: IEEE Computer Society, 2008, pp. 130–137, doi: 10.1109/CCGRID.2008.87.
- [19] S. Jain, R. Kaleem, M. Gamell Balmana, A. Langer, D. Durnov, A. Sannikov, and M. Garzaran, "Framework for scalable intra-node collective operations using shared memory," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '18. IEEE, 2018, pp. 374–385, doi: 10.1109/SC.2018.00032.
- [20] H. Zhu, D. Goodell, W. Gropp, and R. Thakur, "Hierarchical collectives in MPICH2," in *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, M. Ropo, J. Westerholm, and J. Dongarra, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 325–326, doi: 10.1007/978-3-642-03770-2_41.
- [21] X. Luo, W. Wu, G. Bosilca, Y. Pei, Q. Cao, T. Patinyasakdikul, D. Zhong, and J. Dongarra, "HAN: A hierarchical autotuned collective communication framework," in *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, 2020, pp. 23–34, doi: 10.1109/CLUSTER49012.2020.00013.
- [22] S. Chunduri, S. Parker, P. Balaji, K. Harms, and K. Kumaran, "Characterization of MPI usage on a production supercomputer," in *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '18. IEEE, 2018, pp. 386–400, doi: 10.1109/SC.2018.00033.
- [23] J. L. Träff, "Optimal broadcast schedules in logarithmic time with applications to broadcast, reduction, all-broadcast and all-reduction," *ACM Trans. Parallel Comput.*, vol. 12, no. 3, Jun. 2025, doi: 10.1145/3735139.
- [24] M. Barnett, L. Shuler, R. van de Geijn, S. Gupta, D. G. Payne, and J. Watts, "Interprocessor collective communication library (InterCom)," in *Proceedings of IEEE Scalable High Performance Computing Conference*. IEEE, 1994, pp. 357–364, doi: 10.1109/SHPCC.1994.296665.
- [25] E. Chan, M. Heimlich, A. Purkayastha, and R. van de Geijn, "Collective communication: theory, practice, and experience," *Concurrency and Computation: Practice and Experience*, vol. 19, no. 13, pp. 1749–1783, Sep. 2007, doi: 10.1002/cpe.1206.
- [26] T. Hoefler, A. Lumsdaine, and W. Rehm, "Implementation and performance analysis of non-blocking collective operations for MPI," in *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, ser. SC '07. New York, NY, USA: Association for Computing Machinery, 2007, doi: 10.1145/1362622.1362692.
- [27] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed., ser. Springer Texts in Statistics. New York, NY, USA: Springer, 2021, doi: 10.1007/978-1-0716-1418-1.
- [28] S. Hunold and A. Carpen-Amarie, "Reproducible MPI benchmarking is still not as easy as you think," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 12, pp. 3617–3630, Dec. 2016, doi: 10.1109/TPDS.2016.2539167.
- [29] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '15. New York, NY, USA: Association for Computing Machinery, 2015, doi: 10.1145/2807591.2807644.
- [30] M. Han, G. K. Reddy Kuncham, B. Michalowicz, R. Vaidya, M. Abduljabbar, A. Shafi, H. Subramoni, and D. K. Panda, "PML-MPI: A pre-trained ML framework for efficient collective algorithm selection in MPI," in *2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2024, pp. 761–770, doi: 10.1109/IPDPSW63119.2024.00140.